

## บทที่ 6

### การพัฒนาและทดสอบระบบ

ในส่วนของบทนี้จะได้กล่าวถึงขั้นตอนการพัฒนาและทดสอบระบบ ซึ่งเราจะเริ่มตั้งแต่การสร้างในส่วน of ฐานข้อมูลขึ้นมาก่อน แล้วจึงทำการเขียนโปรแกรมซึ่งก็แบ่งออกเป็น 2 ส่วนก็คือส่วนที่เป็นแอปพลิเคชัน ให้สำหรับเจ้าหน้าที่ทะเบียน และส่วนที่เป็นเว็บแอปพลิเคชันสำหรับนักศึกษา หลังจากที่เรเขียนโปรแกรมเสร็จก็จะถึงขั้นตอนการทดสอบระบบ ซึ่งมีรายละเอียดดังนี้

#### 6.1 การสร้างฐานข้อมูล

ในการสร้างฐานข้อมูลนั้นเราอ้างอิงจากการนำเอาโครงสร้างของคลาสต่าง ๆ ในระบบมาสร้างเป็นโครงสร้างตารางฐานข้อมูล และสร้างเป็นตัวฐานข้อมูลใน DB2 ซึ่งมีขั้นตอนดังนี้

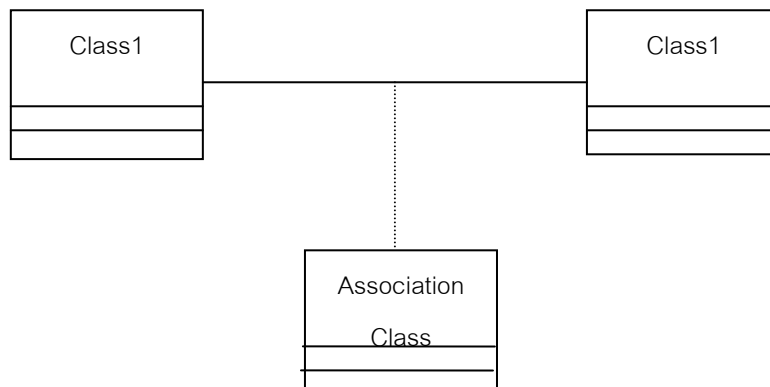
6.1.1 นำเอาโครงสร้างของคลาสมาแปลงเป็นโครงสร้างตารางฐานข้อมูล(Database Schema) ซึ่งมีวิธีการดังนี้

##### การแปลงจาก คลาสไปเป็นตาราง

มีอยู่ 4 ทางที่จะแปลงจาก คลาสเป็น ตารางคือ one-to-one ,one-to-many,many-to-one many-to-many ซึ่งอาจจะแปลงได้หลากหลายขึ้นอยู่กับเหตุผลทางด้านประสิทธิภาพ,ความปลอดภัย,ความสะดวกในการค้นหา

การแปลงนั้นจะยึดหลักของฐานข้อมูลเชิงสัมพันธ์ (relational database) คือมีความสัมพันธ์แบบ many-to-many ,subtype,supertype

ความสัมพันธ์แบบ many-to-many จะต้องทำให้กลายเป็นความสัมพันธ์แบบ one-to-many โดยการสร้างตารางความสัมพันธ์ใหม่



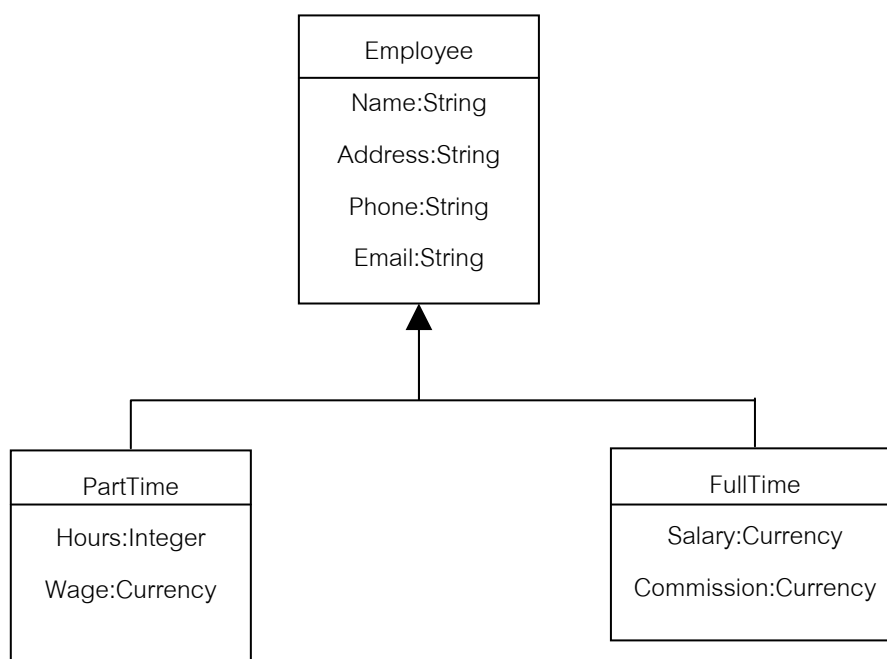
รูป ความสัมพันธ์ของคลาส

เราจะใช้หลักการของ entity-relationship(ER) ช่วยสำหรับความสัมพันธ์ระหว่างคลาส  
เมื่อเราจะแปลงคลาสไปเป็นตารางเรามีอยู่ 3 ตัวเลือกคือ

1. หนึ่งตารางต่อหนึ่งคลาส
2. หนึ่งตารางต่อ คอนกรีตคลาส(concrete class)
3. หนึ่งตารางต่อไฮลาซี(hierarchy)

หนึ่งตารางต่อหนึ่งคลาสนั้น ไม่ยุ่งยากแต่ละคลาสแปลงมาเป็นตารางที่เหมือนกันได้โดยตรง

หนึ่งตารางต่อ คอนกรีตคลาส(concrete class) เรียกว่า “rolling down” คือจาก ตาราง supertype ไปยัง ตาราง subtype ของมัน หนึ่งตารางต่อไฮลาซี(hierarchy) เรียกว่า “rolling up” คือจาก subtype ไปยัง supertype เราจะนำ แอททริบิวท์(attribute) ใน subtype และ supertype คลาสมาแปลงเป็นคอลัมน์ในตารางเดียวและจะมีคอลัมน์ใหม่เพิ่มขึ้นมาในตารางเพื่อเป็นตัวระบุถึง subtype เดิม



*Logical design*

| Table Employee                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PK emp_id:INTEGER<br>emp_name:VARCHAR(25)<br>emp_address: VARCHAR(45)<br>emp_phone: VARCHAR(10)<br>emp_email: VARCHAR(45)<br>emp_salary:DOUBLE PRECISION<br>emp_commission:DOUBLE PRECISION<br>emp_hours:INTEGER<br>emp_wage: DOUBLE PRECISION<br>emp_type:CHAR(1) |

### ***Database Design***

จากตัวอย่างในรูปมีคลาส Employee เป็นคลาสแม่ และมี Parttime ,Fulltime เป็นลูก โดยแปลงมาเป็นตาราง Employee 1 ตารางแต่จะมีการสร้างคอลัมน์ใหม่ขึ้นมาในตารางที่มีชื่อว่า Emp\_type เป็นตัวบ่งบอกว่า เป็น Employee ประเภทใดซึ่งจะทำให้ไม่ต้องเสียเวลาในการค้นหา 3 ตารางในการดูข้อมูลของ Employee

### **การแปลง แอททริบิวต์ ไปเป็น คอลัมน์**

มีหลายทางที่จะแปลงจากแอททริบิวต์เป็นคอลัมน์แต่นั้นไม่มีผลในการแปลงมาเป็นคอลัมน์มันอาจจะ มีผลในการแปลงจาก คลาสมาเป็นตาราง เราอาจจะมี แอททริบิวต์ในคลาสแต่อาจจะไม่มีอยู่ในฐานข้อมูลหรือ คอลัมน์ก็ได้

โปรแกรมประยุกต์อาจจะมีการคำนวณของยอดขายทั้งหมดแต่นั้นไม่จำเป็นว่ายอดขายทั้งหมดจะอยู่ในฐานข้อมูล โดย แอททริบิวต์ ที่ได้มาจากการคำนวณของข้อมูลในฐานข้อมูลจะถูกเรียกว่า “derived attribute” และจะไม่อยู่ในฐานข้อมูล

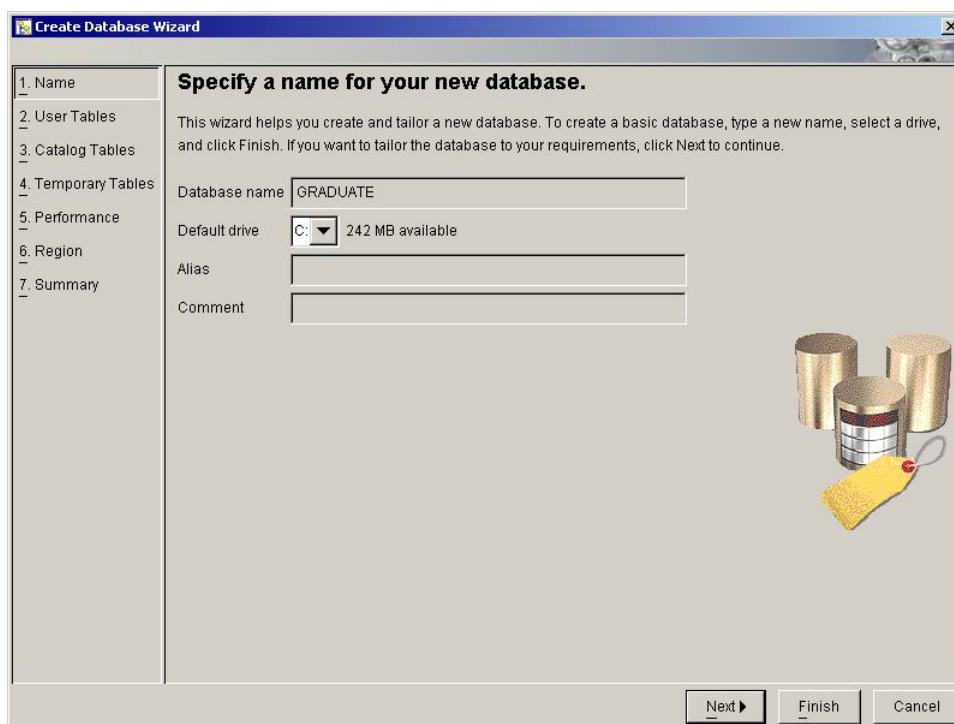
6.1.2 นำโครงสร้างตารางฐานข้อมูลที่ได้ไปทำการนอร์มัลไลซ์(Normalization)เราจะทำการนอร์มัลไลซ์จนโครงสร้างตารางฐานข้อมูลอยู่ในนอร์มัลฟอร์มที่ห้า(Fifth Normal Form) ซึ่งสามารถดูรายละเอียดของตารางที่ได้ในภาคผนวก ข

| COURSE    |              |                                    |
|-----------|--------------|------------------------------------|
| CO#       | SMALLINT     | รหัสหลักสูตร (PK) (Generate)       |
| ENAME     | VARCHAR(100) | ชื่อหลักสูตรภาษาอังกฤษ             |
| TNAME     | VARCHAR(100) | ชื่อหลักสูตรภาษาไทย                |
| FT_DEGREE | VARCHAR(100) | ชื่อเต็มปริญญาภาษาไทย              |
| FE_DEGREE | VARCHAR(100) | ชื่อเต็มปริญญาภาษาอังกฤษ           |
| ST_DEGREE | VARCHAR(100) | ชื่อย่อปริญญาภาษาไทย               |
| SE_DEGREE | VARCHAR(100) | ชื่อย่อปริญญาภาษาอังกฤษ            |
| DEGREE    | VARCHAR(1)   | ปริญญาโท / เอก (M:Master,D:Doctor) |
| MAJOR#    | VARCHAR(2)   | รหัสสาขาวิชา (FK)                  |
| FACT#     | VARCHAR(2)   | รหัสคณะ (FK)                       |
| DEPT#     | VARCHAR(2)   | รหัสภาควิชา (FK)                   |
| MINOR#    | VARCHAR(2)   | รหัสแขนงวิชา (FK)                  |

ตารางที่ 6-1 ตัวอย่างตารางที่ได้มาจากการแปลงคลาสไดอะแกรมมาเป็นโครงสร้างตาราง

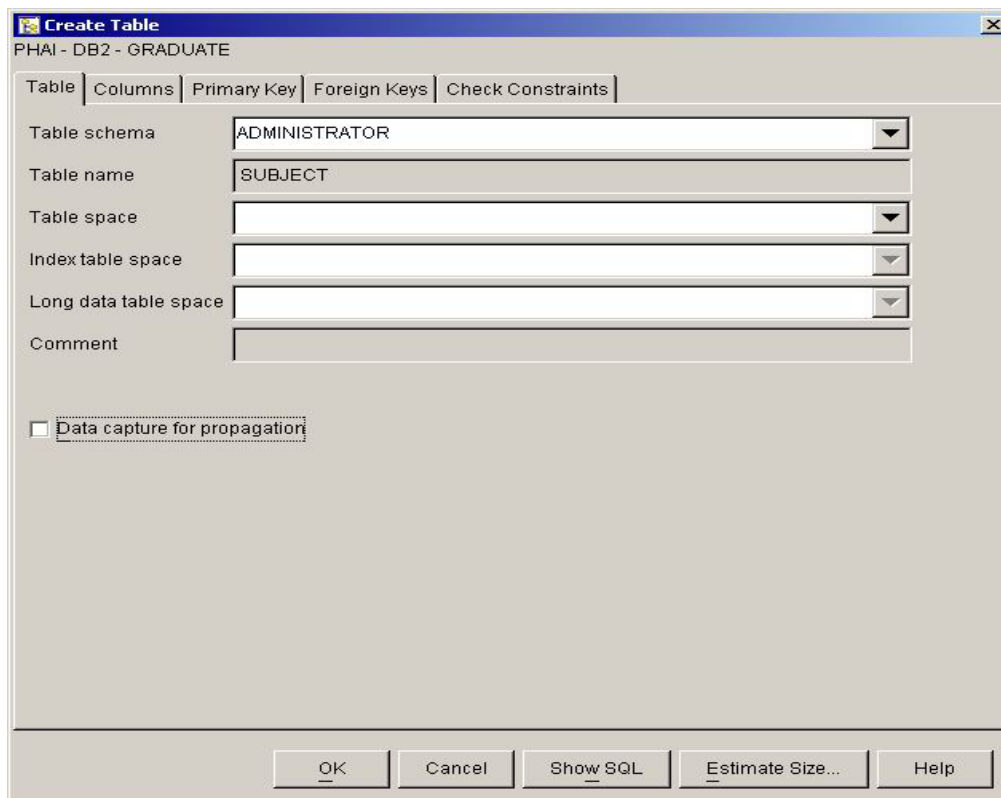
6.1.3 กำหนดรายละเอียดของชนิดข้อมูล(Data Type) ของแต่ละแอตทริบิวต์โดยอ้างอิงตามชนิดข้อมูลของ DB2

6.1.4 ทำการสร้างฐานข้อมูลบน DB2 ขึ้น โดยใช้ โปรแกรม Control Center



รูปที่ 6-1 การสร้างฐานข้อมูลบน DB2 โดยใช้โปรแกรม Control Center

### 6.1.5 ทำการสร้างตารางแต่ละตาราง บนฐานข้อมูล ที่เราสร้างขึ้น โดยใช้โปรแกรม Control Center



รูปที่ 6-2 การใช้ Control Center เพื่อการสร้างตาราง

6.1.6 ทำการกรอกข้อมูลบางอย่างเพื่อเป็นตัวอย่างในการทดลองเขียนโปรแกรมโดยใช้โปรแกรม Command Center

6.1.7 ในการติดต่อกับฐานข้อมูลต่าง ๆ เราก็จะใช้โปรแกรม Command Center ซึ่งสามารถทำงานด้วยภาษาเอสคิวแอล(SQL)ได้ และโปรแกรม Control Center ซึ่งจะเป็นตัวจัดการในการบริหารฐานข้อมูล

หลังจากทั้ง 7 ขั้นตอนนี้เราก็จะมีฐานข้อมูลอยู่บนเครื่องเซิร์ฟเวอร์ที่เราสามารถติดต่อเข้ามาใช้งานได้จากเครื่องคอมพิวเตอร์อื่นได้โดยผ่านทางโพรโทคอลที่ซีพีทีบีไอพี(TCP/IP Protocol)

## 6.2 การเขียนโปรแกรมในส่วนของแอปพลิเคชัน

แอปพลิเคชันที่สร้างขึ้นนี้จากข้างต้นที่เคยกล่าวแล้วว่าใช้ภาษาจาวา ซึ่งเป็นภาษาที่มีข้อดีหลายประการและรองรับการพัฒนาโปรแกรมแบบ Object Oriented ได้เป็นอย่างดี รวมถึงใช้โปรแกรม Forte for java เป็นเครื่องมือในการพัฒนา ในการเขียนโปรแกรมนั้นค่อนข้างมีรายละเอียดมากมาย และสามารถหาความรู้

โหลดได้ฟรีจากเว็บไซต์ของซัน ผู้พัฒนาจะพยายามทำให้เนื้อหาที่ง่ายที่สุดในการทำความเข้าใจโดยการแบ่งเป็นข้อ ๆ ตามลำดับขั้นตอนดังนี้

6.2.1 ออกแบบยูสเซอร์อินเทอร์เฟซ(User Interface) โดยแบ่งโปรแกรมออกเป็น ส่วน ๆ ตามยูสเคส ในแต่ละยูสเคสก็นำมาแบ่งเป็นส่วนย่อย ๆ อีกตามซีเควนซ์ไดอะแกรม โดยให้แต่ละส่วนเป็นพาเนล(Panel)

6.2.2 หลังที่ทำการแบ่งเป็นพาเนลได้แล้วนั้น แต่ละพาเนลก็หมายถึงหนึ่งคลาสนั่นเอง เราจึงสามารถแยกแต่ละพาเนลไปเขียนโปรแกรม อย่างอิสระ แล้วจึงค่อยนำมารวมกันในภายหลัง

6.2.3 ในการเขียนโปรแกรมติดต่อกับฐานข้อมูลนั้นเราใช้ JDBC เป็นตัวเชื่อมต่อให้โดยมีการระบุถึงเซิร์ฟเวอร์ฐานข้อมูลด้วยหมายเลขไอพี(IP Address)

6.2.4 หลังจากขั้นตอนนี้เราสามารถแยกแต่ละส่วนมาเขียนโปรแกรมได้ ซึ่งแยกไปตามคลาสต่าง ๆ

try

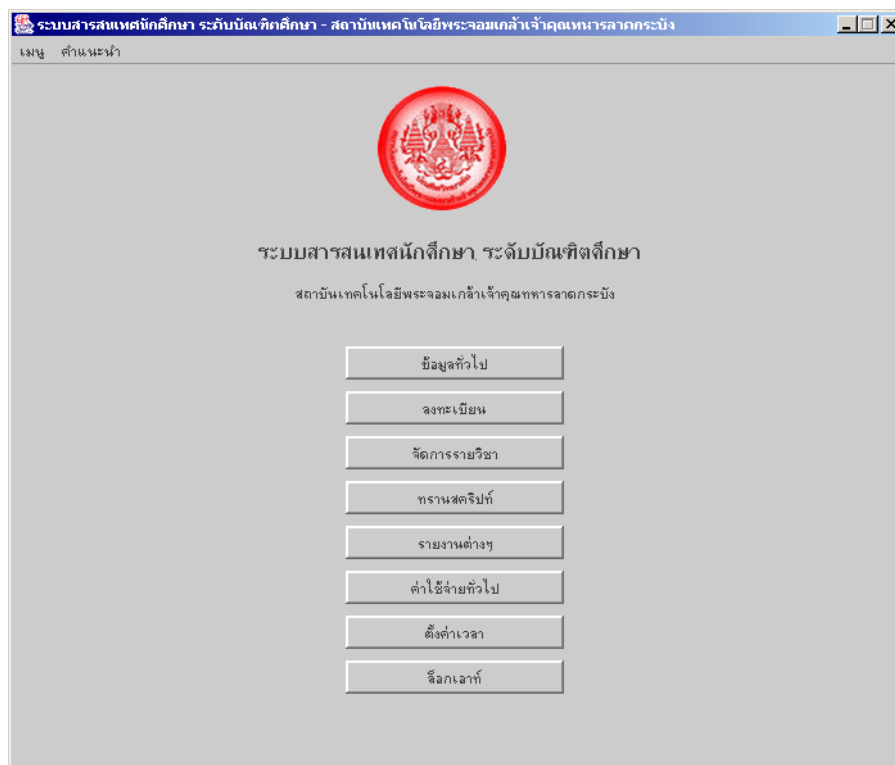
```
{
    Class.forName("COM.ibm.db2.jdbc.net.DB2Driver").newInstance();
    con = DriverManager.getConnection(url, uname, passwd);
}
catch(ClassNotFoundException cnfex)
{
    System.err.println("Failed to load JDBC/ODBC driver.");
    cnfex.printStackTrace();
}
catch(SQLException sqllex)
{
    System.err.println("Unable to connect");
    sqllex.printStackTrace();
}
catch(Exception ex)
{
    ex.printStackTrace();
}
```

#### ตัวอย่างโค้ดการติดต่อฐานข้อมูลด้วย JDBC

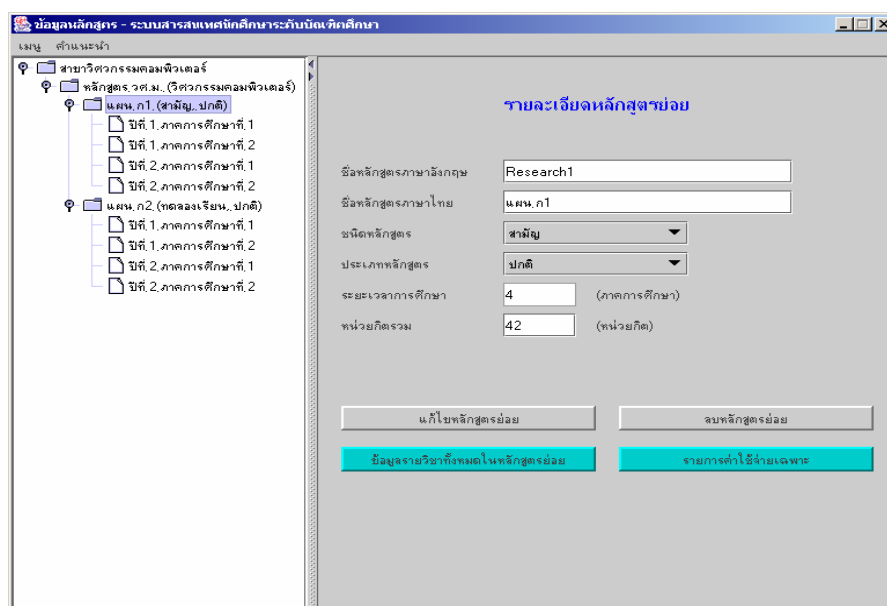
6.2.5 สิ่งหนึ่งที่ต้องคำนึงในการเขียนโปรแกรมก็คือข้อกำหนดต่าง ๆ ที่ได้ทำการศึกษาจากระบบจริง ซึ่งเราต้องเขียนโปรแกรมให้มีการตรวจสอบตรงตามข้อกำหนดเหล่านั้นด้วย ยกตัวอย่างเช่นนักศึกษาในระดับบัณฑิตศึกษานั้นสามารถเข้ามานักศึกษาใหม่ได้ทั้งในภาคเรียนที่ 1 และภาคเรียนที่ 2 ดังนั้นการจัดแบ่งกลุ่ม

นักศึกษาจึงต้องแบ่งตามภาคและปีที่เข้า ไม่สามารถแบ่งตามรหัสนักศึกษาแต่เพียงอย่างเดียวได้เหมือนในระดับปริญญาตรี

6.2.6 เมื่อเราเขียนโปรแกรมแต่ละส่วนเสร็จก็จะนำมาประกอบในส่วนของพาแนลหลัก



รูปที่ 6-3 รูปส่วนอินเทอร์เฟซของ MainPanel



รูปที่ 6-4 รูปส่วนอินเทอร์เฟซของ รายละเอียดหลักสูตร

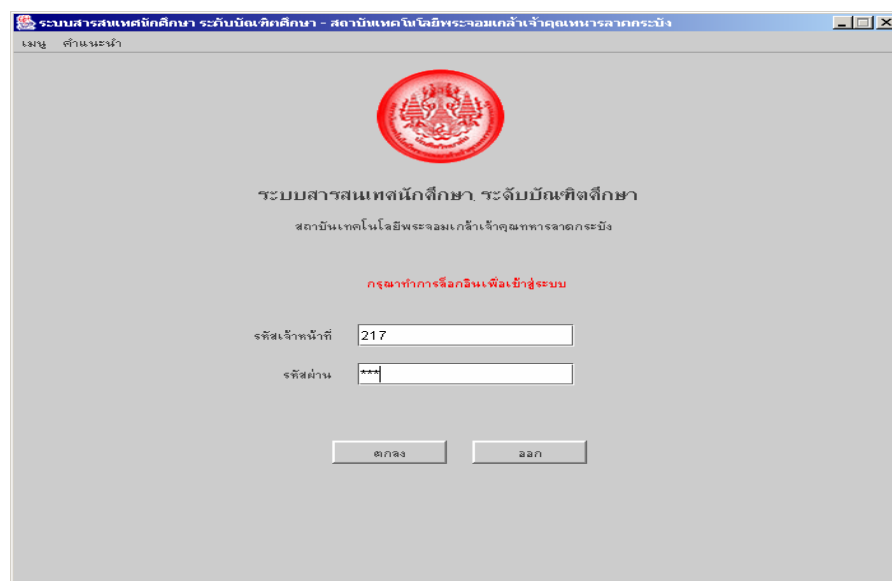


จากขั้นตอนข้างต้นอาจไม่สามารถอธิบายลงในรายละเอียดทั้งหมดได้ซึ่งสามารถดูในรายละเอียดจากซอสโค้ด(Source Code)

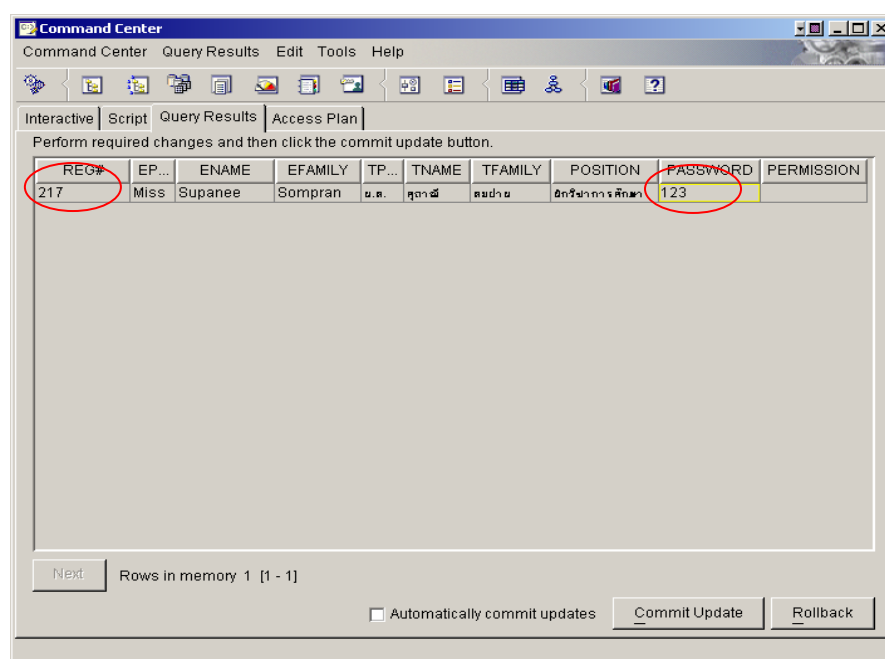
### 6.3 การทดสอบโปรแกรมในส่วนของแอปพลิเคชัน

การทดสอบนี้จะทดสอบในกรณีของโครงสร้างของหน่วยงานการศึกษา ดังขั้นตอนต่อไปนี้

1. ล็อกอินเข้าสู่ระบบ ดังรูป โดย รหัสเจ้าหน้าที่ และ รหัสผ่านได้จากตาราง Registrar ในฐานข้อมูลของ DB2



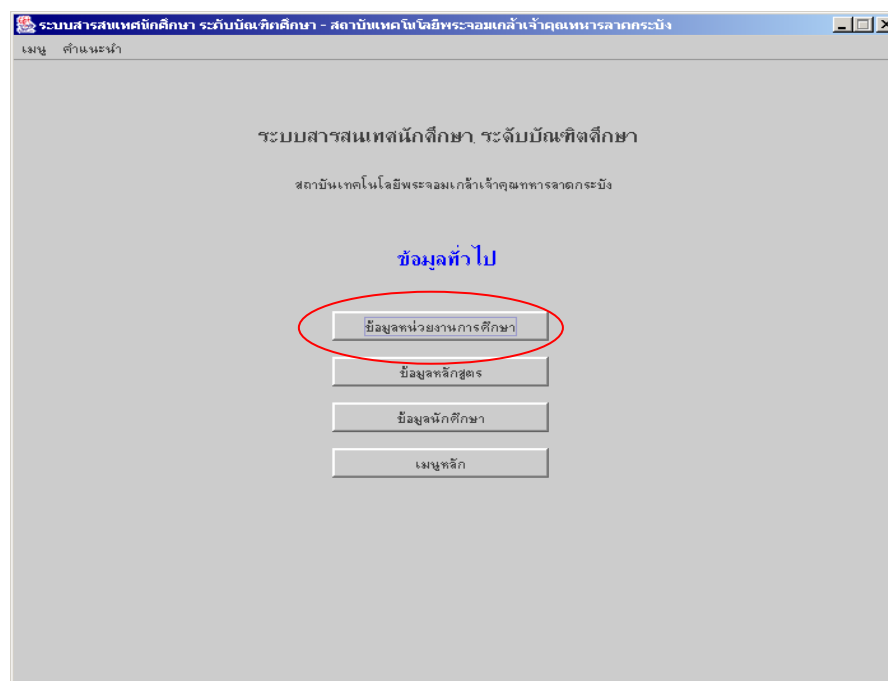
รูปที่ 6-5 แสดงหน้าจอการล็อกอิน



| REG# | EP... | ENAME   | EFAMILY | TP... | TNAME  | TFAMILY | POSITION    | PASSWORD | PERMISSION |
|------|-------|---------|---------|-------|--------|---------|-------------|----------|------------|
| 217  | Miss  | Supanee | Sompran | ม.ศ.  | ศุภาณี | สมชาย   | อธิการศึกษา | 123      |            |

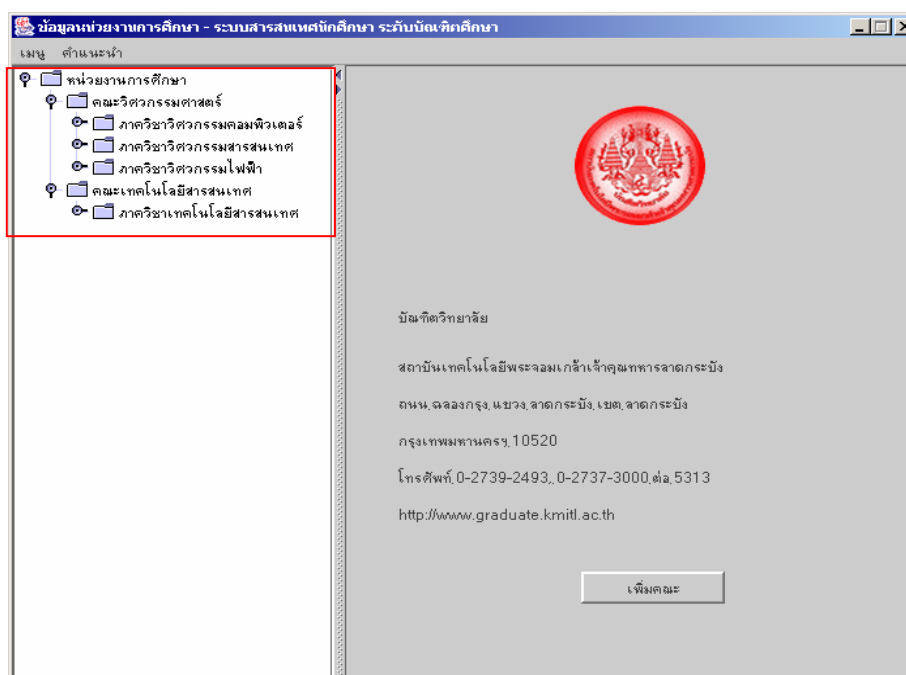
รูปที่ 6-6 รูปแสดงข้อมูลที่อยู่ในฐานข้อมูล

## 2. ในหมวดของข้อมูลทั่วไปเลือกเมนูข้อมูลหน่วยงานการศึกษา



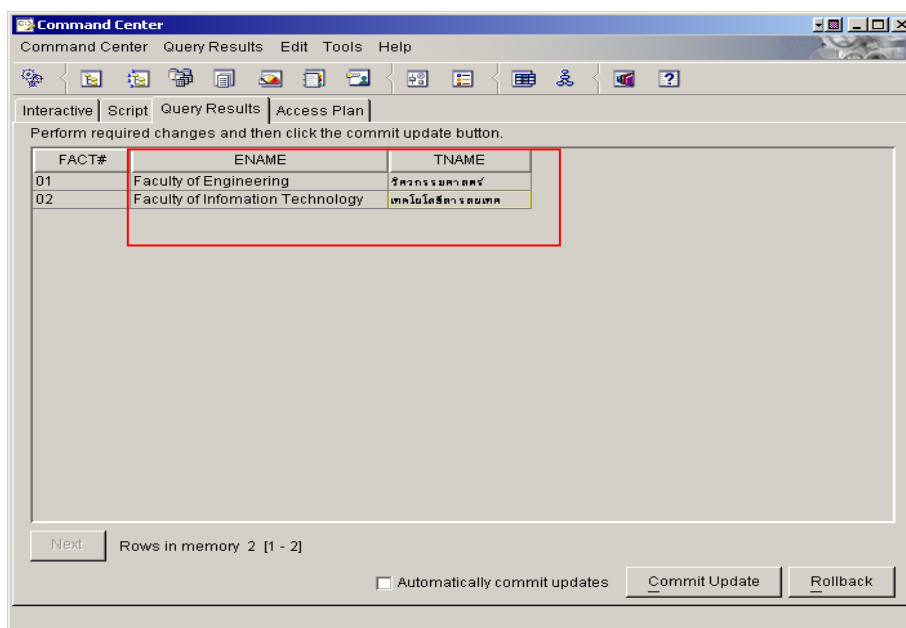
รูปที่ 6-7 แสดงการเลือกเมนู ข้อมูลหน่วยงานศึกษา

## 3. จะได้นี้หน้าตาของโครงสร้างของข้อมูลหน่วยงานการศึกษา ลักษณะของทรีโมเดลดังรูป



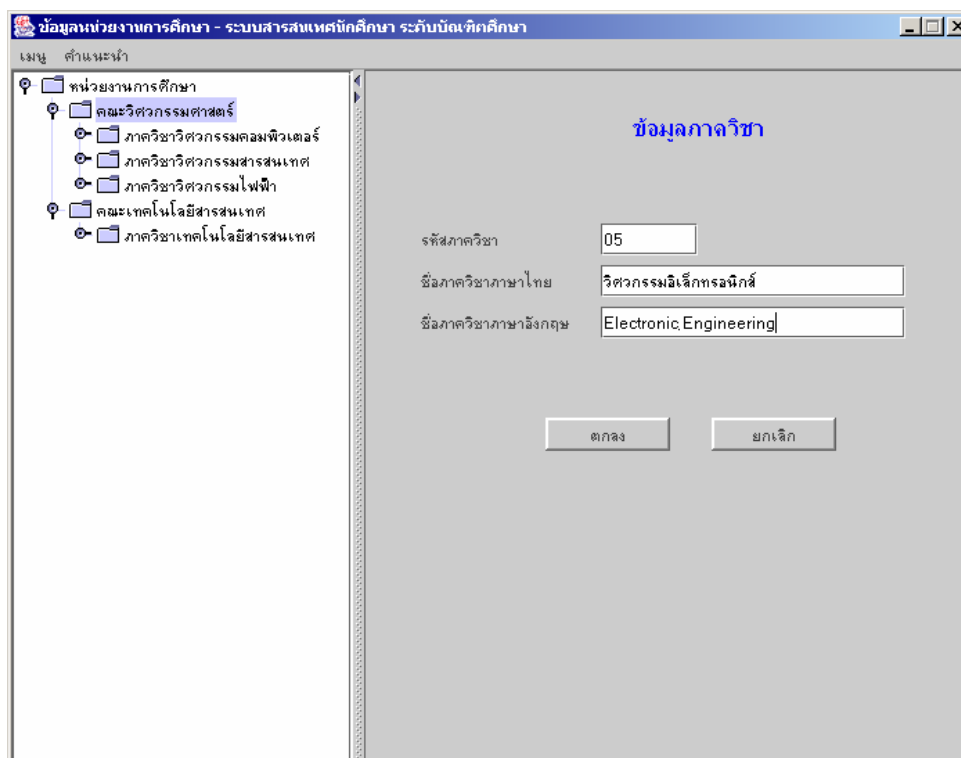
รูปที่ 6-8 แสดงข้อมูลของคณะและภาควิชา

4. ลองเทียบกับข้อมูลในตาราง Faculty ใน Command Center ของ db2 จะพบว่ามี 2 คณะเท่านั้นดังรูป



รูปที่ 6-9 แสดงข้อมูลในตาราง Faculty

5. ทำการทดสอบการเพิ่มภาควิชาในคณะวิศวกรรมศาสตร์ โดยกดปุ่มเพิ่มภาควิชา จะได้ช่องใส่ข้อมูลดังรูปที่



รูปที่ 6-10 แสดงการเพิ่มภาควิชา

6. ทำการใส่ข้อมูลลงไป แล้วกดปุ่มตกลง
7. ทดสอบดูกับในฐานข้อมูลของตาราง Department จะได้ดังรูป

รูปที่ 6-11 แสดงผลการเพิ่มภาควิชา

8. ทดสอบการเพิ่มสาขาวิชาในภาควิชาวิศวกรรมอิเล็กทรอนิกส์ ดังรูป

รูปที่ 6-12 แสดงการเพิ่มสาขาวิชา

ข้อมูลหน่วยงานการศึกษา - ระบบสารสนเทศศึกษาระดับบัณฑิตศึกษา

เมนู หน้าแรก

หน่วยงานการศึกษา

- คณะวิศวกรรมศาสตร์
  - ภาควิชาวิศวกรรมคอมพิวเตอร์
  - ภาควิชาวิศวกรรมสารสนเทศ
  - ภาควิชาวิศวกรรมไฟฟ้า
  - ภาควิชาวิศวกรรมอิเล็กทรอนิกส์
  - สาขาวิชาวิศวกรรมอิเล็กทรอนิกส์** (highlighted)
- คณะเทคโนโลยีสารสนเทศ
  - ภาควิชาเทคโนโลยีสารสนเทศ

ข้อมูลสาขาวิชา

รหัสสาขาวิชา: 05

ชื่อสาขาวิชาภาษาไทย: วิศวกรรมอิเล็กทรอนิกส์

ชื่อสาขาวิชาภาษาอังกฤษ: Electronic Engineering

ปุ่ม: เพิ่มแผนผังวิชา, แก้ไขข้อมูลสาขาวิชา, ลบข้อมูลสาขาวิชา

สาขาวิชาที่เพิ่มเข้ามา

รูปที่ 6-13 แสดงผลการเพิ่มสาขาวิชา

9. หลังจากเพิ่มข้อมูลสาขาวิชา แล้วลองเทียบกับตาราง Major ในฐานข้อมูลดู

Command Center

Command Center Query Results Edit Tools Help

Interactive Script Query Results Access Plan

Perform required changes and then click the commit update button.

| MAJOR# | ENAME                   | TNAME                  | DEPT# |
|--------|-------------------------|------------------------|-------|
| 01     | Computer Engineering    | วิศวกรรมคอมพิวเตอร์    | 01    |
| 02     | Information Engineering | วิศวกรรมสารสนเทศ       | 02    |
| 03     | Information Technology  | เทคโนโลยีสารสนเทศ      | 03    |
| 04     | Electrical Engineering  | วิศวกรรมไฟฟ้า          | 04    |
| 05     | Electronic Engineering  | วิศวกรรมอิเล็กทรอนิกส์ | 05    |

สาขาวิชาที่เพิ่มในฐานข้อมูล

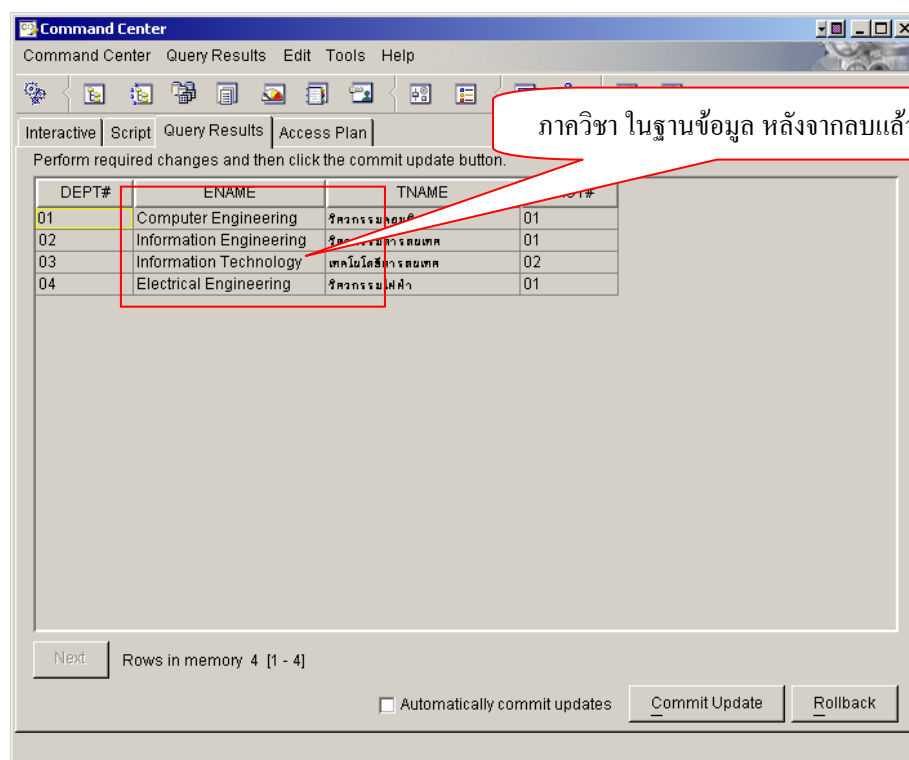
Next Rows in memory 5 [1 - 5]

☐ Automatically commit updates Commit Update Rollback

รูปที่ 6-14 แสดงข้อมูลหลังจากการเพิ่มสาขาวิชา

10. ลองทดสอบการลบสาขาวิชา วิศวกรรมอิเล็กทรอนิกส์โดยการกดปุ่ม ลบข้อมูลสาขาวิชา

11. หลังจากนั้นจะลบภาควิชา วิศวกรรมอิเล็กทรอนิกส์ได้โดยกดปุ่มลบภาควิชา



รูปที่ 6-15 แสดงข้อมูลในตาราง *Department*

จะเห็นว่าการทดสอบ Application Program ในส่วนของข้อมูลหน่วยงานการศึกษา เป็นไปได้ถูกต้องตามกระบวนการที่ได้ออกแบบไว้ ในส่วนอื่นๆก็สามารถทดสอบได้ในกรณีเดียวกัน ซึ่งยกตัวอย่างไว้เพียงส่วนของข้อมูลการศึกษาเท่านั้น

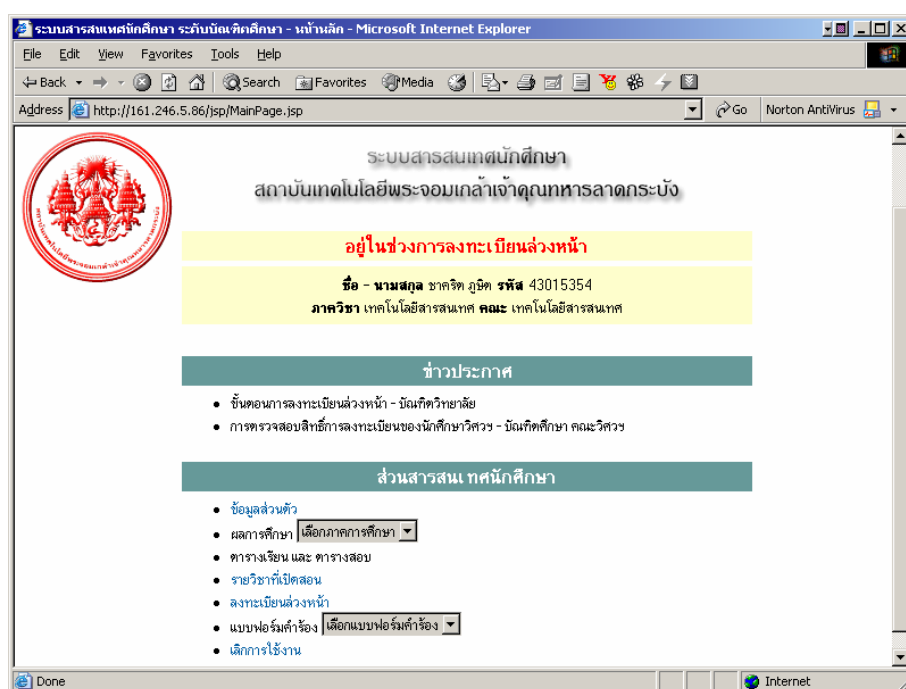
## 6.4 การเขียนโปรแกรมในส่วนของเว็บแอปพลิเคชัน

การเขียนโปรแกรมในส่วนนี้จะแตกต่างจากการเขียนโปรแกรมในส่วนแอปพลิเคชันเพราะว่า การแสดงผลในส่วนนี้จะอยู่บนเว็บเบราว์เซอร์(Web Browser) ดังนั้นการเขียนโปรแกรมแบบนี้จึงอยู่บนสิ่งแวดล้อมอินเทอร์เน็ตโดยใช้เทคโนโลยีของ Servlet, JSP และ JavaBean ซึ่งได้กล่าวถึงทฤษฎีทั้งสองเรื่องนี้แล้วในบทที่ 3 แล้ว

6.4.1 เนื่องจากโปรแกรมในส่วนนี้จะอยู่บนเว็บเบราว์เซอร์ เราจึงต้องมี เว็บเซิร์ฟเวอร์(WebServer) ที่รองรับเทคโนโลยีของ Servlet และ JSP ซึ่งในโครงงานนี้เราใช้ Apache TOMCAT

6.4.2 เราสามารถเขียนโปรแกรมของเราทั้งในส่วนคลาสที่เป็น Servlet, JSP และ JavaBean ไว้ในโฟลเดอร์(Folder) ที่ Apache TOMCAT สร้างไว้ให้ เราก็จะสามารถทดลองโปรแกรมที่เราเขียนได้โดยผ่านทางเว็บเบราว์เซอร์บนเครื่องเราเอง

6.4.3 ขั้นตอนต่อไปก็คือการออกแบบยูสเซอร์อินเทอร์เฟซในแต่ละส่วน แล้วก็สามารถนำไปเขียนโปรแกรม โดยแยกจากกันเพื่อทดลองทีละส่วนได้



รูปที่ 6-16 การทดลองใช้งานผ่านเว็บเบราว์เซอร์

6.4.4 ในขั้นตอนนี้เราสามารถเขียนคลาสต่าง ๆ ขึ้นมาใช้งานได้ตามปกติ การติดต่อกับฐานข้อมูลก็ทำคล้ายกับการติดต่อในส่วนแอปพลิเคชัน แต่คลาสที่เราสร้างขึ้นเพื่อนำไปใช้ควรสร้างให้เป็น JavaBean เพื่อจะมีความสามารถสูงสุดในการนำไปใช้ใน Servlet และ JSP

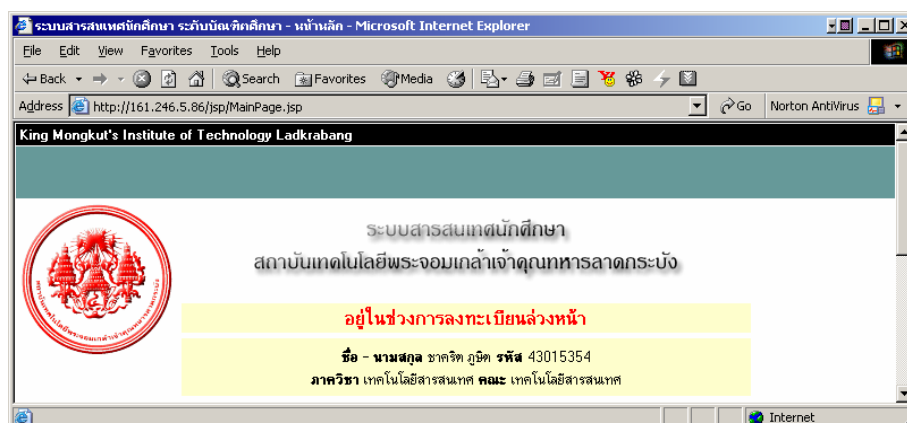
6.4.5 นอกจากใช้ Forte for java แล้วในการเขียน JSP ซึ่งเป็นการทำงานแบบ Server Side Included อย่างหนึ่งจำเป็นต้องสร้างโครงร่างของหน้า html ขึ้นมาก่อนซึ่งในที่นี้เราใช้ Macromedia Dreamweaver เข้ามาช่วยเสริมการทำงานในส่วนนี้ด้วย

6.4.6 นำส่วนต่าง ๆ มาประกอบกันให้สามารถทำงานเชื่อมต่อกันได้ด้วยการกำหนดลิ้งค์(Link) ให้เชื่อมโยงถึงกัน

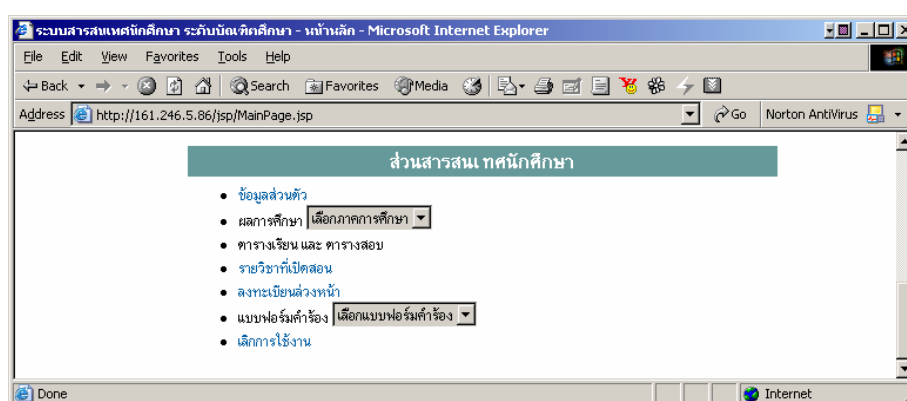
## 6.5 การทดสอบโปรแกรมในส่วนของเว็บแอปพลิเคชัน

การทดสอบในส่วนของเว็บแอปพลิเคชันนั้น เพื่อให้เข้าใจในการทำงานของระบบ ในที่นี้จะยกตัวอย่างการลงทะเบียนล่วงหน้าของนักศึกษา

6.5.1 การลงทะเบียนล่วงหน้า นั้น เป็น 1 ในเมนูที่มีการเปลี่ยนแปลงตามเวลาปัจจุบัน กล่าวคือ หากวันที่นักศึกษามีการล็อกอินเข้ามาอยู่ในช่วงของการลงทะเบียนล่วงหน้า เมนูการลงทะเบียนล่วงหน้าก็จะสามารถทำงานได้(Active) ซึ่งจะสามารถตรวจสอบช่วงเวลาได้จาก แถบแสดงเวลา

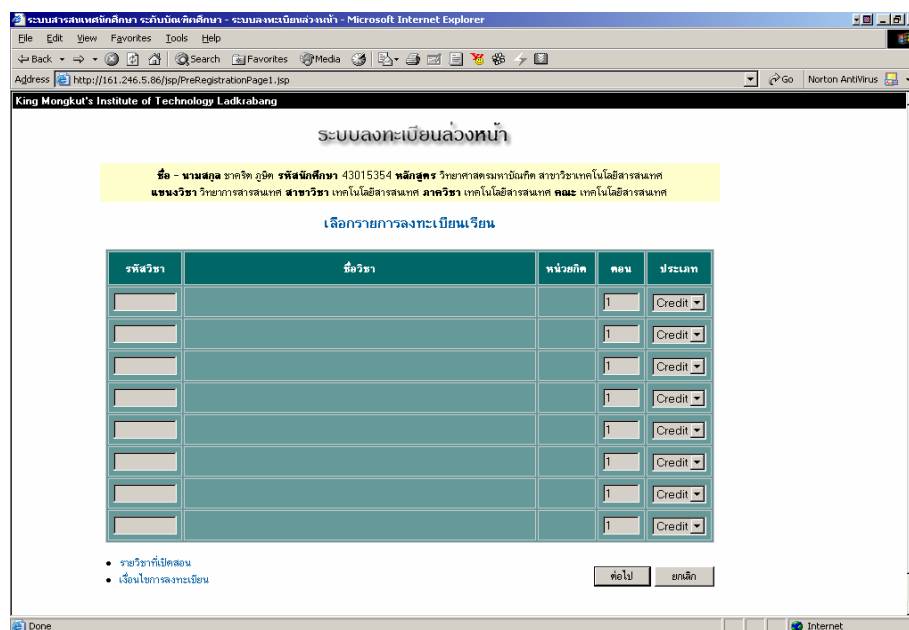


รูปที่ 6-17 แสดงช่วงเวลาปัจจุบัน



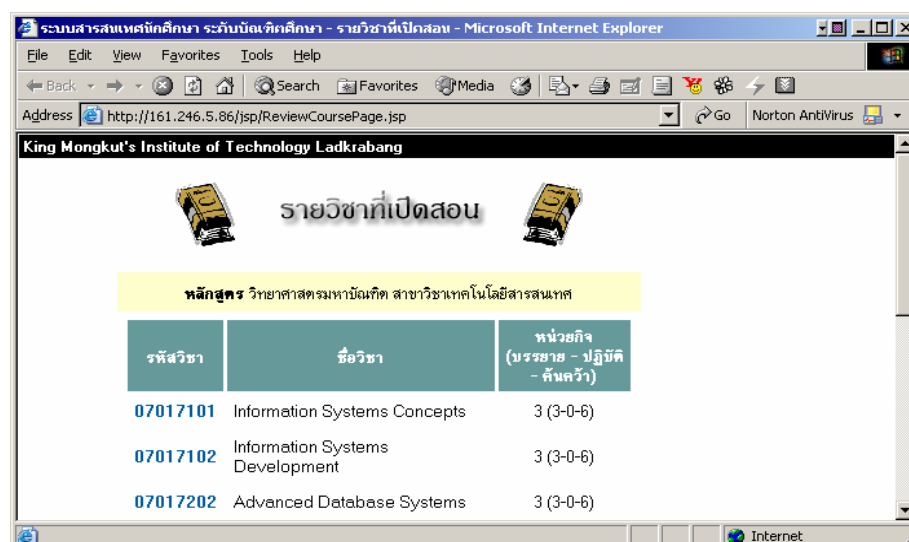
รูปที่ 6-18 แสดงเมนูที่เปลี่ยนแปลงตามช่วงเวลา



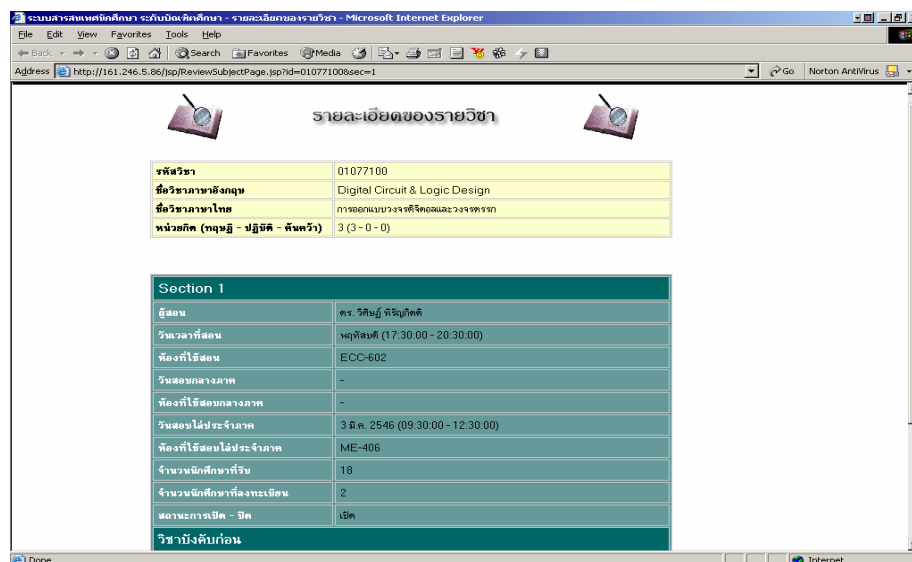


รูปที่ 6-19 แสดงให้เลือกลงทะเบียนตามรายวิชาที่เปิดสอนตามหลักสูตร

6.5.2 นักศึกษาจะทำการเลือกลงทะเบียนรายวิชาโดยป้อนรหัสวิชา ตามรายวิชาที่เปิดสอนตามหลักสูตรโดยนักศึกษาสามารถตรวจสอบรายชื่อวิชาที่เปิดสอนได้จากเมนู รายชื่อวิชาที่เปิดสอน โดยนักศึกษาสามารถเข้าไปดูรายละเอียดของแต่ละรายวิชา คำอธิบายรายวิชา และดูว่าวิชาดังกล่าวมีวิชาบังคับก่อนหรือไม่ หรือคุณสมบัติของรายวิชาว่าเปิด หรือปิด สามารถดูจำนวนนักศึกษาที่ลงทะเบียนไปแล้ว และจำนวนนักศึกษาที่รับสูงสุดได้ ในกรณีที่รายวิชามีการจำกัดจำนวนผู้ที่ลงทะเบียนรายวิชาดังกล่าวไว้



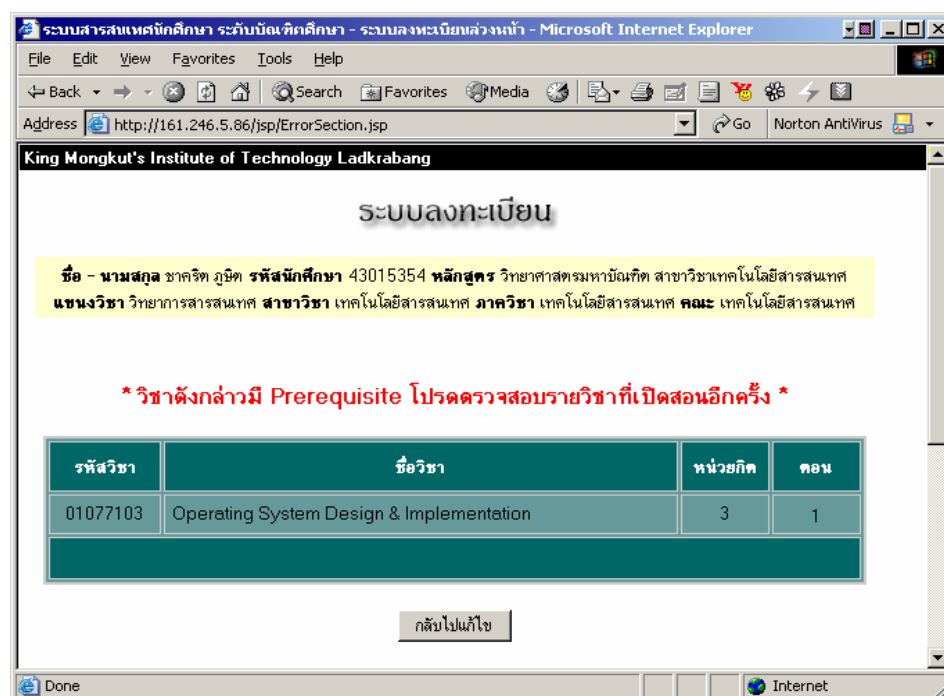
รูปที่ 6-20 แสดงรายวิชาที่เปิดสอนตามหลักสูตร



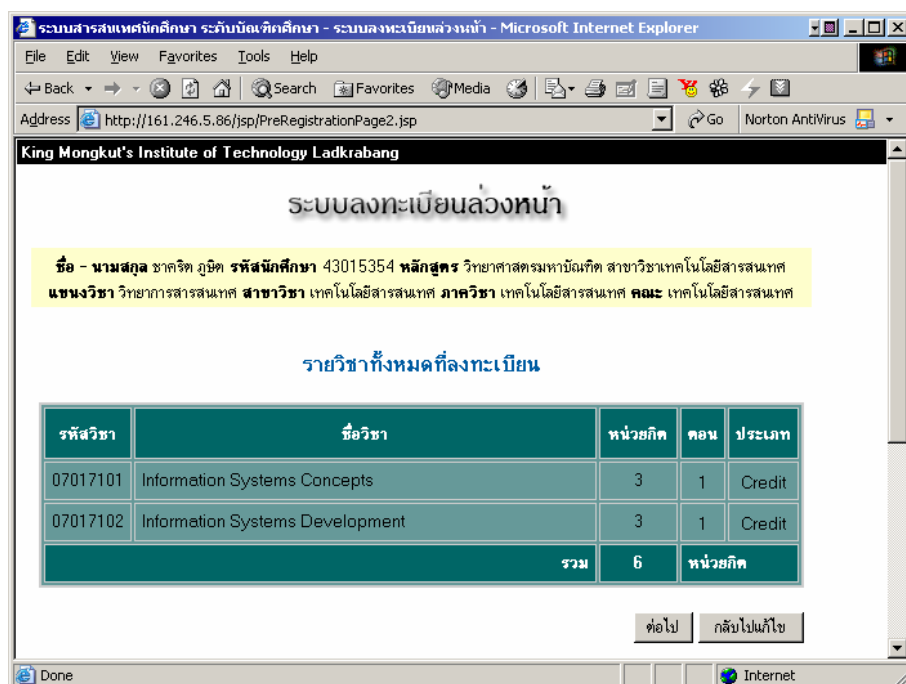
รูปที่ 6-21 แสดงรายละเอียดรายวิชาที่เปิดสอนตามหลักสูตร

6.5.3 ระบบจะทำการตรวจสอบเงื่อนไขการลงทะเบียนต่างๆ เช่น วิชาที่จะลงทะเบียนมีวิชาบังคับก่อนหรือไม่ เคยลงทะเบียนเรียนวิชาดังกล่าวมาแล้วหรือยัง และจำนวนหน่วยกิตรวมที่สามารถลงทะเบียนได้ รวมถึงรายวิชาที่มีการจำกัดจำนวนผู้ที่ลงทะเบียน ถ้าไม่ผ่านการตรวจสอบก็ไม่สามารถลงทะเบียนวิชาดังกล่าวได้ ระบบก็จะทำการแสดงรายวิชาที่ลงทะเบียนอีกครั้ง พร้อมทั้งแสดงรายละเอียดค่าใช้จ่ายทั้งหมดให้นักศึกษาทราบ

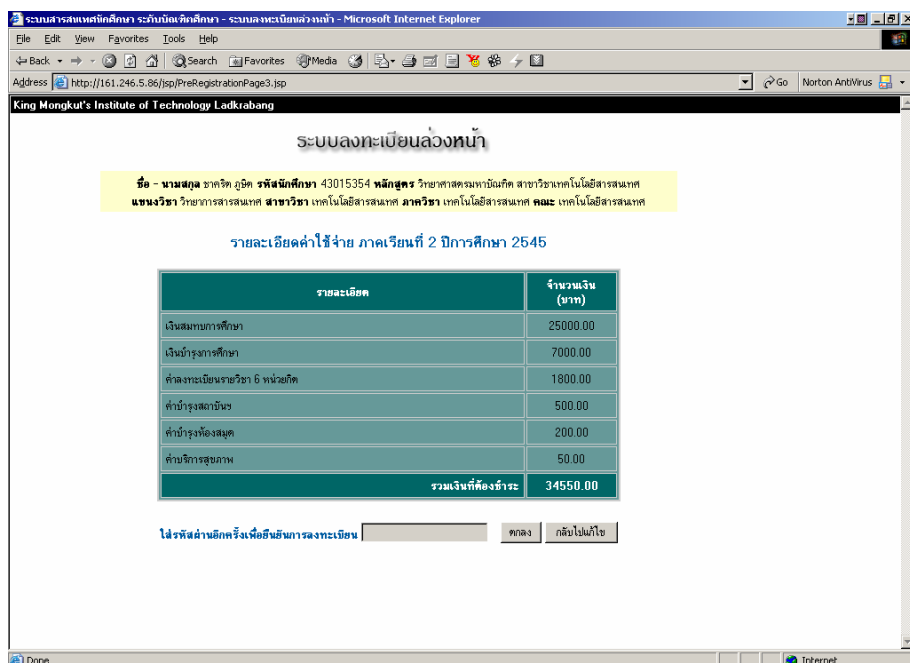
6.5.4 ขั้นตอนสุดท้าย ระบบจะทำการเก็บข้อมูลการลงทะเบียนนี้ไว้เพื่อนำไปทดสอบกับการลงทะเบียนจริง และแสดงข้อความเพื่อบอกช่วงเวลาที่นักศึกษาต้องมาลงทะเบียนจริงอีกครั้ง



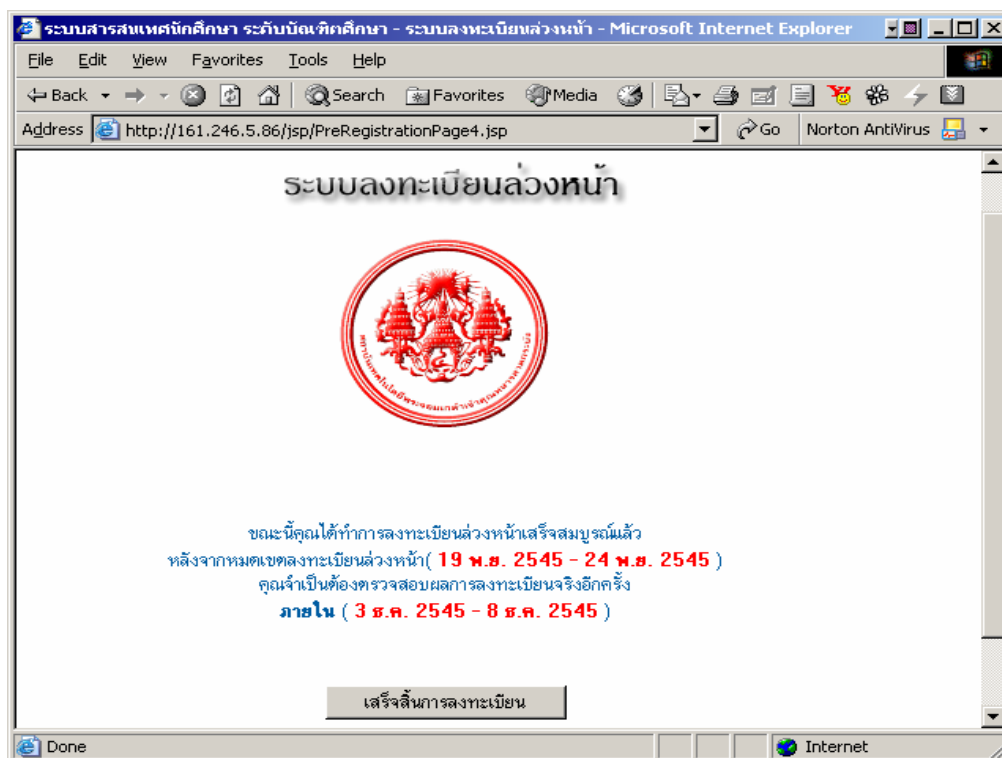
รูปที่ 6-22 แสดงระบบตรวจสอบวิชาที่ต้องเรียนวิชาบังคับก่อน



รูปที่ 6-23 แสดงรายวิชาที่ลงทะเบียน



รูป 6-24 แสดงค่าใช้จ่ายตามหลักสูตร



รูปที่ 6-25 แสดงข้อความการลงทะเบียนสมบูรณ์